Private Set Intersection

Carsten Baum & Tyge Tiessen

1 Background

A Private Set Intersection (PSI) protocol allows two parties $\mathcal{S}, \mathcal{C}$ who each as input hold a set X, Y respectively, to compute and learn $I = X \cap Y$ and nothing else. In particular, $\mathcal{S}$ never learns any information about $Y \setminus I$ while $\mathcal{C}$ learns nothing about $X \setminus I$. In the following, we will call $\mathcal{S}$ the *server* and $\mathcal{C}$ the *client*.

There are many protocols that solve this problem cryptographically with varying degrees of efficiency, and they are sometimes optimized for special cases. One such special case is *mobile contact discovery*: assume that $\mathcal{S}$ is the server of a popular messaging app, and $\mathcal{C}$ wants to learn which of its contacts is registered with the app. It could upload all its contacts to $\mathcal{S}$ but that would harm the client's privacy. Alternatively, $\mathcal{S}$ could send a list of all its users to $\mathcal{C}$ but this would harm the privacy of all users of the app who are not contacts of $\mathcal{C}$. As can be seen, this scenario is ideal for the use of PSI. It is also an interesting scenario, as $|X| \gg |Y|$ is to be expected. This is often called *unbalanced* or *asymmetric* PSI.

One particularly simple construction is due to Baldi et al. [BBDC$^+$11], and we will use the description of [RA18] of the protocol. For it, assume that there is a fixed finite abelian group $\mathbb{G}$ of prime order p, meaning $|\mathbb{G}| = p$. The group must be such that a version of the so-called Diffie-Hellman problem is hard, which is usually true for subgroups of $\mathbb{Z}_q$ for very large q or so-called elliptic curve groups. We assume the existence of two cryptographic hash functions:

- H maps strings from $\{0,1\}^{512}$ to elements from $\mathbb{G}$.
- H' maps group elements from $\mathbb{G}$ to strings from $\{0,1\}^{256}$.

Then, the protocol works as follows:

1. $\mathcal{S}$ has an input $X = \{x_1, \ldots, x_n\}$, while $\mathcal{C}$ has an input $Y = \{y_1, \ldots, y_m\}$. We assume $x_i, y_j \in \{0,1\}^{512}$.

2. $\mathcal{S}$ chooses a uniformly random $\alpha \in \mathbb{Z}_p^*$ and computes

$$\Omega_i = \mathsf{H}'(\mathsf{H}(x_i)^\alpha)$$

for all $i \in \{1, \ldots, n\}$. It then places all Ω_i on a list L, shuffles L and sends it to $\mathcal{C}$.

3. Next, $\mathcal{C}$ chooses $\beta_1, \ldots, \beta_m \in \mathbb{Z}_p^*$ uniformly at random, computes $\Theta_j = (\mathsf{H}(y_j))^{\beta_j}$ for all $j \in \{1, \ldots, m\}$ and sends $\Theta_1, \ldots, \Theta_m$ to $\mathcal{S}$.

4. Upon receiving $\Theta_1, \ldots, \Theta_m$, $\mathcal{S}$ computes $T_j = (\Theta_j)^\alpha$ for all $j \in \{1, \ldots, m\}$ and sends $T_1, \ldots, T_m$ to $\mathcal{C}$.

5. Upon receiving $T_1, \ldots, T_m$, $\mathcal{C}$ checks if $\mathsf{H}'((T_j)^{1/\beta_j}) \in L$ for each $j \in \{1, \ldots, m\}$. For any such element in L it says that y_j is in the intersection, otherwise not.

2 Tasks

In this project, you will implement the aforementioned PSI protocol. The implementation should use the Ristretto Elliptic Curve group and can be done in Rust, Go, C or Javascript, based on the existing implementations of Ristretto in these languages. The project has the following tasks:

1. Install and experiment with the Ristretto library `https://ristretto.group/` and how it can be used to implement the PSI protocol. Towards implementing H you should use Elligator (which is also provided by the existing implementations of Ristretto), and you can implement H' by hashing any serialized form of a group element using e.g. SHA-256.

2. Implement the PSI protocol. For this, you should have methods that emulate the operations of the client and the server and that obtain their respective sets X, Y as input. The communication could either be implemented via multithreading in the same process, or by implementing multiple processes that communicate via the local loopback interface.

3. Benchmark the runtime and the communication of your solution. How does it change as the size of X or Y increases? In particular, how does it change as you increase X but keep Y the same?

4. What happens if $\mathcal{S}$ in step 4 of the protocol uses a different value α' instead of α as the exponent for Θ_j. Establish, both using mathematics and by making changes your implementation, the outcomes on the protocol correctness of this modification.

Towards making such changes in step 4 impossible, we will assume that $\mathcal{S}$ computes $R = g^\alpha$ in step 2. Let H'' be a hash function that maps arbitrary strings $\{0,1\}^*$ to $\mathbb{Z}_p$. We additionally define the following two algorithms which form a so-called Non-Interactive Zero-Knowledge proof:

Prove: On input $\alpha, \Theta_1, \ldots, \Theta_m$ choose $a \in \mathbb{Z}_p^*$ uniformly at random and compute $A = g^a, A_1 = (\Theta_1)^a, \ldots, A_m = (\Theta_m)^a$. The compute

$$e \leftarrow \mathsf{H}''(A, A_1, \ldots, A_m, \Theta_1, \ldots, \Theta_m, R)$$

as well as $z = a + e \cdot \alpha \bmod p$ and output $\pi = (A, A_1, \ldots, A_m, z)$.

Verify: On input $\pi = (A, A_1, \ldots, A_m, z)$ as well as $R, \Theta_1, \ldots, \Theta_m, T_1, \ldots, T_m$ recompute $e \leftarrow \mathsf{H}''(A, A_1, \ldots, A_m, \Theta_1, \ldots, \Theta_m, R)$ and check that $g^z = A \cdot R^e$ as well as $(\Theta_j)^z = A_j \cdot (T_j)^e$ for every $j \in \{1, \ldots, m\}$.

The new PSI protocol then works as follows: Then, the protocol works as follows:

1. $\mathcal{S}$ has an input $X = \{x_1, \ldots, x_n\}$, while $\mathcal{C}$ has an input $Y = \{y_1, \ldots, y_m\}$. We assume $x_i, y_j \in \{0,1\}^{512}$.

2. $\mathcal{S}$ chooses a uniformly random $\alpha \in \mathbb{Z}_p^*$ and computes

$$\Omega_i = \mathsf{H}'(\mathsf{H}(x_i)^\alpha)$$

for all $i \in \{1, \ldots, n\}$. It additionally computes $R = g^\alpha$, places all Ω_i on a list L, shuffles L and sends L, R to $\mathcal{C}$.

3. Next, $\mathcal{C}$ chooses $\beta_1, \ldots, \beta_m \in \mathbb{Z}_p^*$ uniformly at random, computes $\Theta_j = (\mathsf{H}(y_j))^{\beta_j}$ for all $j \in \{1, \ldots, m\}$ and sends $\Theta_1, \ldots, \Theta_m$ to $\mathcal{S}$.

4. Upon receiving $\Theta_1, \ldots, \Theta_m$, $\mathcal{S}$ computes $T_j = (\Theta_j)^\alpha$ for all $j \in \{1, \ldots, m\}$ as well as **Prove** which outputs π. It then sends $T_1, \ldots, T_m, \pi$ to $\mathcal{C}$.

5. Upon receiving $T_1, \ldots, T_m, \pi$, $\mathcal{C}$ first runs the **Verify** algorithm defined above. If **Verify** rejects, then abort. Otherwise, $\mathcal{C}$ checks if $\mathsf{H}'((T_j)^{1/\beta_j}) \in L$ for each $j \in \{1, \ldots, m\}$. For any such element in L it says that y_j is in the intersection, otherwise not.

Finally, do the following task:

6. Implement the modified PSI protocol and check if the previously described attack still works. In addition, can you modify the new protocol, in particular the choice of the element R, so that a version of the attack still works?